

Hydra: Phase-Aware Workload Characterization of LLM Inference across Edge SoC Generations, Backends, and Quantization Levels

Amir Taherin¹ Sana Taghipour Anvari¹ Charles Amante¹ Yixiao Chen¹ Ruben Noroian¹ Zlatan Feric¹ Nicolas Bohm Agostini¹ Pu Zhao¹ José Cano² Bin Ren³
Yanzhi Wang¹ David Kaeli¹

¹Northeastern University ²University of Glasgow ³William & Mary

IEEE International Symposium on Workload Characterization (IISWC) 2026



Available



Reviewed



Reproducible

Introduction: LLMs are arriving on edge SoCs

- Personal assistants, robots, vehicles, and cameras increasingly run LLMs **on-device**:
 - Privacy
 - Offline operation
 - Bounded latency
- **Edge SoC**: CPU + GPU on one die, **shared memory**, tight power and thermal budgets
- LLM inference has two phases:
 - **Prefill** (once, large GEMMs)
 - **Decode** (autoregressive decoding, small GEMVs)

The deployment question: which backend, precision, platform, and at what energy cost?



Xavier (2018)



Orin (2022)



Thor (2025)

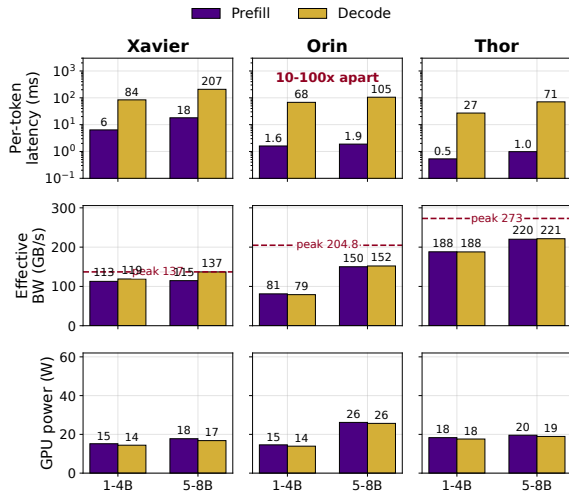
	Xavier	Orin	Thor
GPU	Volta	Ampere	Blackwell
CPU	8-core Carmel	12-core A78AE	14-core Neoverse V3AE
Memory	32 GB	32 GB	128 GB
Peak BW (GB/s)	137	204.8	273
Power budget	30 W	60 W	130 W
JetPack	5	6	7

Three consecutive NVIDIA Jetson AGX generations

Inference backends		Quant.
 Hugging Face	Transformers (Python)	~
 llama.cpp	GGML (C++)	✓
 LLM	PyTorch (Python)	✓
 TGI	Transformers (Rust)	✓
 MACHINE LEARNING COMPIRATION	TVM (C++)	✓

Formats: FP32, FP16, BF16, **quantized** Q8, Q6, Q4, ...

Motivation: Aggregate latency hides edge LLM deployment requirements



- One metric (“latency”, “throughput”) mixes **prefill**, **decode**, and **orchestration**, each stressing the SoC differently
- **Performance**: per-token rate differs **10–100×** between phases
- **Resource utilization**: the same phase loads each generation’s memory system differently, relative to its peak
- **Efficiency**: power tracks neither latency nor bandwidth; the fastest configuration is not the most efficient

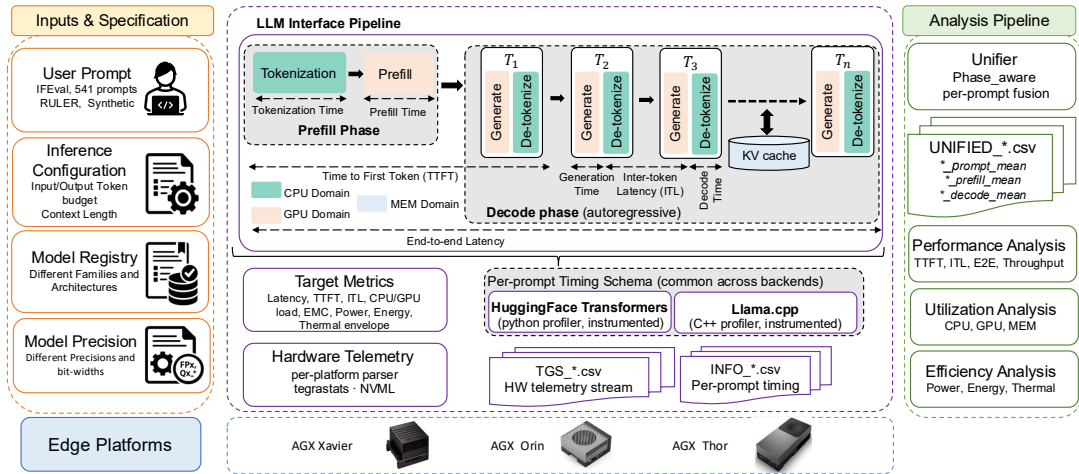
Gap: no common, phase-aware measurement schema across backends, precisions, and SoC generations.

Prior work measures pieces; none connects them

		Hardware		Backend		Models			Coverage				Reporting		Artifact			
Work		Edge	Cross-gen.	llama.cpp	HF	Comm. sch.	# models	# families	Cross-arch	Precision	Timing	HW perf.	Efficiency	Phase	Per-prompt	Tool	Dataset	# metrics
Edge	Dhar	yes	no	yes	no	no	1	1	no	part	part	part	no	no	no	no	no	5
	Nezami	yes	no	yes	no	no	8	7	yes	part	part	part	no	part	no	yes	no	4
	PalmBench	yes	yes	yes	no	no	10	8	yes	full	part	full	part	part	no	yes	no	7
	LLMPi	yes	no	yes	no	no	7	4	no	full	part	no	part	no	no	no	no	6
	Husom	yes	no	yes	no	no	4	3	no	part	part	no	part	no	part	yes	no	4
	Jang & Mor.	yes	no	–	–	–	17	4	no	–	part	no	part	no	no	no	no	4
	EdgeProf.	no	no	no	no	no	4	4	no	part	part	no	part	no	no	yes	no	4
	MELT	yes	no	yes	no	no	7	5	yes	part	part	full	full	full	yes	yes	no	7
Server	ServeGen	no	no	–	–	no	10	3	no	–	part	no	no	no	yes	yes	yes	4
	TokenPow.	no	no	no	yes	no	15	4	yes	part	part	no	part	full	part	yes	no	5
	TokenSim	no	no	no	no	no	2	2	no	part	part	no	no	part	no	yes	no	2
	LLMSrvSim	no	no	no	no	no	6	2	no	–	no	part	no	part	no	yes	no	2
	LLM-IB	no	no	yes	no	no	8	3	yes	part	part	no	part	part	no	yes	yes	5
	Hydra (ours)	yes	yes	yes	yes	yes	13	7	yes	full	full	full	full	full	yes	yes	yes	16

Hydra provides a holistic view that is missing in prior work: one canonical record across backends, precisions, and SoC generations, with the corpus released.

Hydra: one canonical record from backend timing + SoC telemetry



Backend, precision, model, and platform become *comparable*, not separate benchmark runs.

Main characterization

- **Edge Hardware:**
 - 3 generations: **Xavier, Orin, Thor**
- **Models:**
 - **13** instruction-tuned LLMs
 - **7** families (1–8B)
- **Backends:**
 - HF Transformers
 - llama.cpp
- **Precision formats:**
 - bf16, F16, Q8_0, Q6_K, Q4_K_M
- **Input/Output length:**
 - Input from IFEval benchmark:
13 to 345 tokens ($\mu = 47, \sigma = 23$)
 - Output fixed: **500** tokens

Input-length sweep

- **RULER** long-context benchmark
- Input: **1k / 3k / 5k** tokens
- Output fixed: **500** tokens
- **Focus:** prefill, TTFT
- Orin + Thor

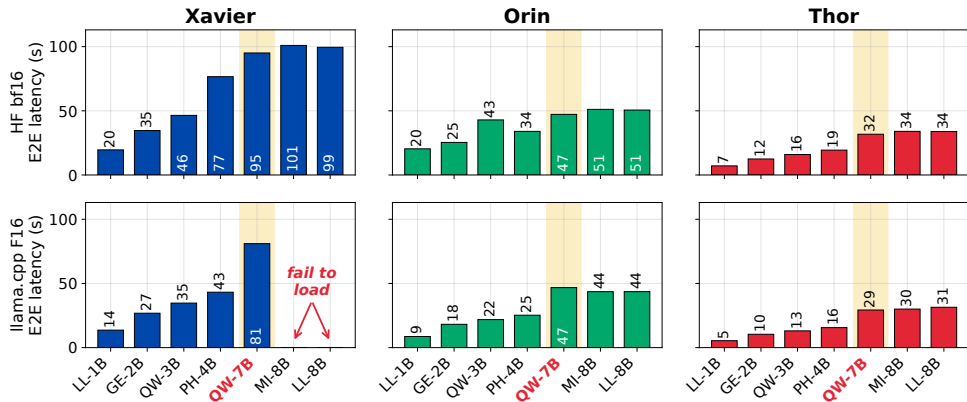
Output-length sweep

- IFEval-filtered prompts
- Input fixed: **~50** tokens
- Output: **1k / 3k / 5k** tokens
- **Focus:** decode throughput
- Orin + Thor



~107K records released

Finding 1: newer SoC generations help, but unevenly

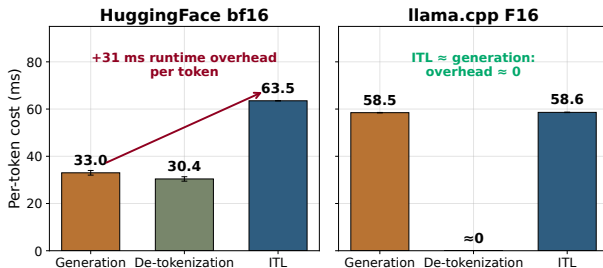


- The gain is **not a constant factor**: it shifts with **model size, architecture, and backend**
- **End-to-end latency drops across generations**: Xavier → Orin → Thor, on both backends
- **Missing bars are findings too**: Xavier cannot run F16 7–8B (memory allocator limit)

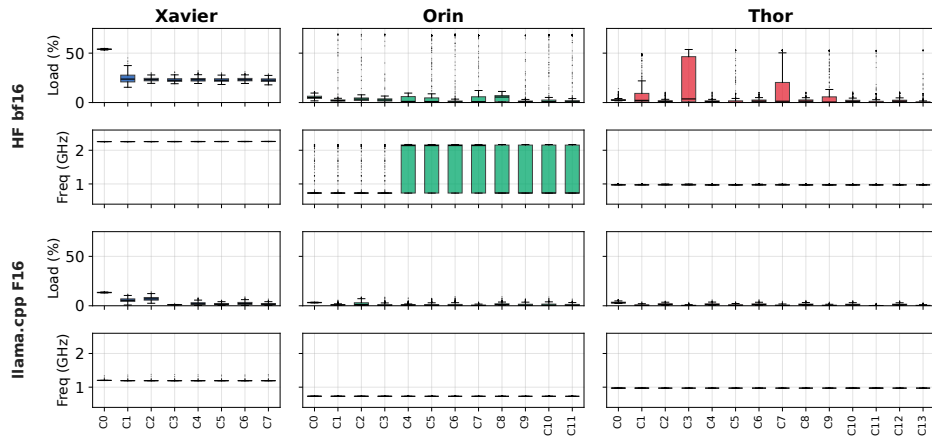
Finding 2: the backend with *slower* kernels wins end-to-end

- llama.cpp is **faster end-to-end** (Thor, Qwen2.5-7B)
- Yet its per-token generation is $\sim 1.8\times$ **slower** than HF
- Why? **Backend structure**: no Python orchestration between token steps
- **Kernel speed** $\neq$ **responsiveness**

Qwen2.5-7B on Thor — per-token cost by stage

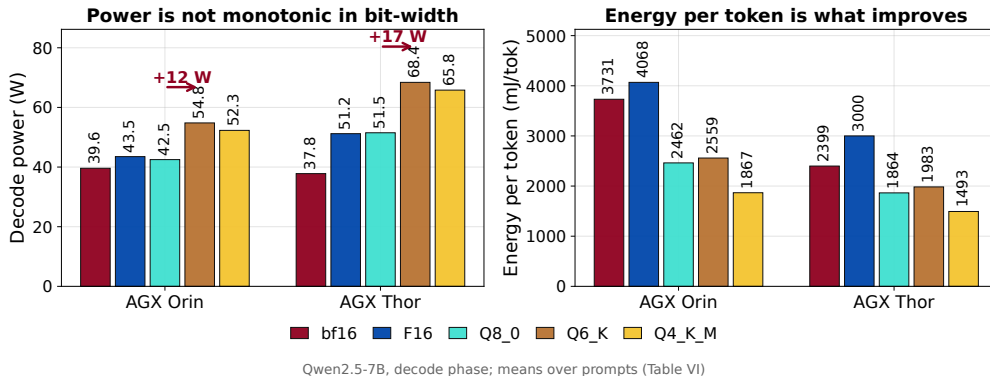


The smoking gun: where does HuggingFace's time actually go?



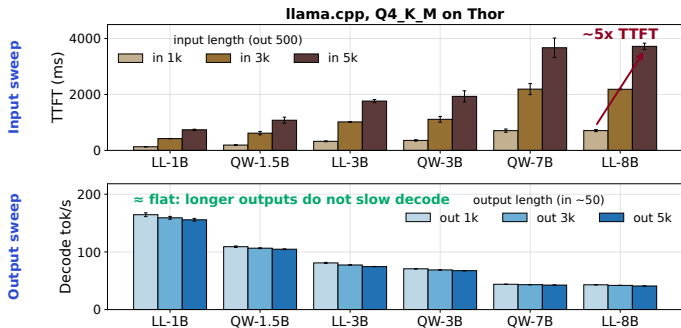
- HF bf16: an **orchestration storm**: several cores driven hard on every token step
- llama.cpp F16: near-idle CPUs: the GPU runs, the CPU waits
- “LLM inference is a GPU workload” is only half the story on an edge SoC

Finding 3: power is not monotonic in bit-width



- **Q6_K draws more power and runs hotter than Q8_0**
 - k-quant unpacking costs compute on every access
- Energy per token still improves with quantization, but **bit-width does not predict power**: judge formats by **mJ/token**, and mind the power cap

Finding 4: input length affects your first token and output length does not slow you down



- **TTFT grows with input length:**
 - Prefill is **compute-bound**, and longer context is paid **up front**
- **Decode throughput stays flat with output length:**
 - Steady-state decode is **memory-bound**
- **Budget rule:**
 - Input length affects TTFT; output length affects time and energy

What does this mean when you deploy?

- **Newer platform? (Finding 1)**

Gains shift with model size, architecture, and backend — and **nominal memory is not deployability**: verify the allocator path, not the spec sheet

- **Latency-bound application? (Finding 2)**

Backend structure dominates: pick the backend that meets your end-to-end latency, not the fastest kernel benchmark

- **Energy budget matters? (Finding 3)**

Choose formats by **mJ/token**, never by nominal bit-width, and check power draw against your cap (Q6_K can exceed it where Q8_0 fits)

- **Long prompts or long outputs? (Finding 4)**

Input length sets first-token latency; **output length** sets total time and energy

**Across the corpus, these choices can change energy per prompt by 1.8–2.4 $\times$.
Lessons from this study can significantly increase the number of prompts served
per energy budget.**

Concluding Remarks

- 1. Phase-aware benchmarking of LLM inference beats end-to-end benchmarking:**
prefill, decode, and orchestration stress an edge SoC differently; one aggregate number hides the real bottleneck
- 2. Four Edge LLM deployment effects that aggregate metrics miss:**
 - a) Backend structure wins,
 - b) Non-monotonic power consumption of precision formats,
 - c) Length-driven phase costs, and
 - d) Allocator-limited deployability
- 3. Code and dataset are released:**
Hydra + $\sim 107\text{K}$ per-prompt records

Thank you — questions?



Paper
[arXiv:2608.25053](https://arxiv.org/abs/2608.25053)



Code & Dataset
github.com/amirtaherin/hydra

taherin.a@northeastern.edu



Backup: the released corpus at a glance

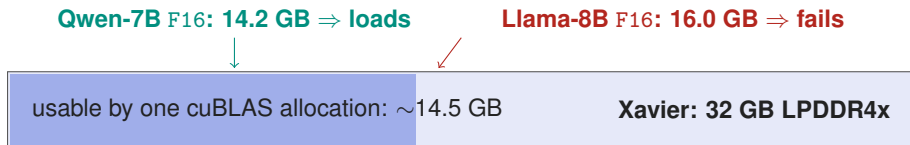
- 3** Jetson generations: Xavier, Orin, Thor
- 13** instruction-tuned LLMs, 7 families
- 5** execution formats: bf16, F16, Q8_0, Q6_K, Q4_K_M
- 2** backends under one schema
- ~107K** per-prompt records, released

Plus input/output-length sensitivity sweeps and IFEval runs



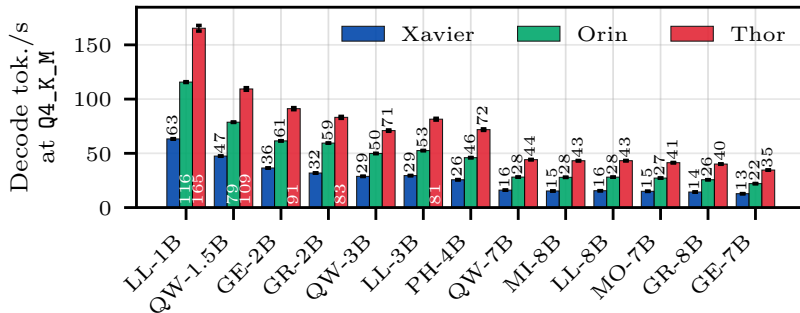
github.com/amirtaherin/hydra
+ Zenodo archival copy — all three IISWC
artifact badges

Backup: deployability is an allocator story, not a capacity story

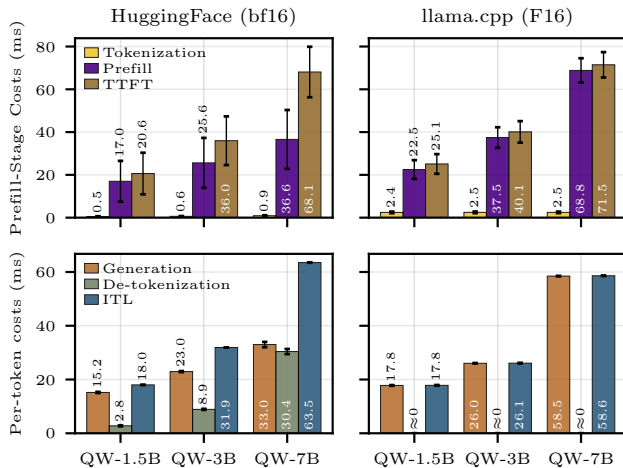


- Xavier has 32 GB — yet several F16 7–8B configurations **cannot run**: the JetPack-5 cuBLAS/NVMAP path caps a single contiguous allocation at ~14.5 GB
- 14.2 GB loads; 16.0 GB does not. **Nominal capacity is not deployability** — allocator behavior and tensor footprint decide
- Invisible in spec sheets; visible the moment measurement is platform-aware

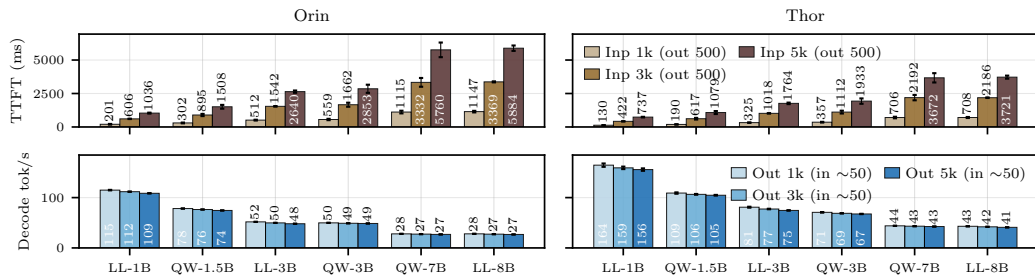
Backup: Q4_K_M decode throughput across generations



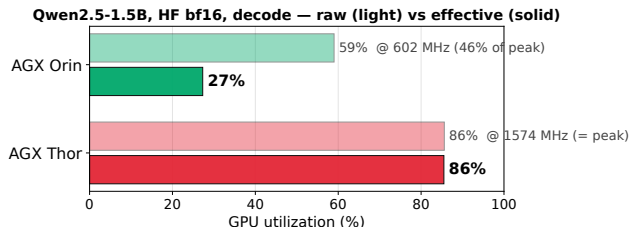
Backup: full phase-level latency decomposition (Qwen family, Thor)



Backup: length sensitivity on Orin and Thor



Backup: utilization numbers do not transfer across generations



GPU effective utilization

$$U_{\text{eff}} = \text{load} \times \frac{f_{\text{GPU}}}{f_{\text{peak}}}$$

- **Raw counters mislead:** Orin reports 59% busy — measured at 602 MHz, 46% of its peak clock (DVFS drops the clock during CPU-side orchestration gaps)
- **Thor pins its GPU clock at peak:** its 86% means what it says
- **Clock-normalized U_{eff} makes generations comparable:** 27% vs. 85% — Orin's real headroom is $\sim 3\times$, invisible in raw counters